



MICROSERVICES COURSE

Session 24 — Phase 3 Final Session

Architecture Clinic #2 + Phase 3 Wrap-Up

Reviewing Every Decision Since Session 1 · Phase 3 Complete · Phase 4 Capstone Preview



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

TIME

3:00–5:30 PM

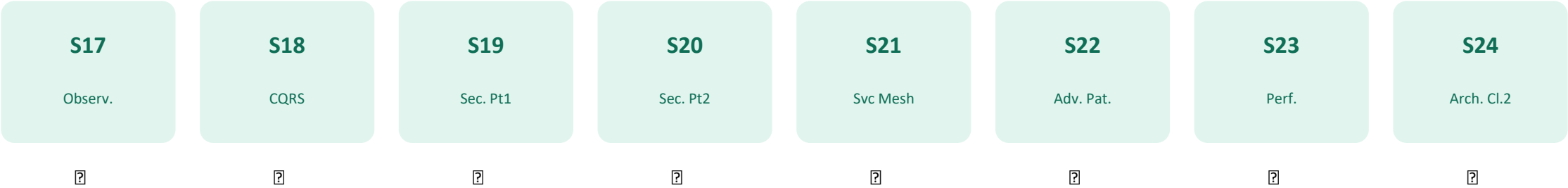
DURATION

2.5 Hours

PHASE

Phase 3 · Final Session

Phase 3 Summary — All Sessions Complete



Today we do not add a new feature. We ask, with 23 sessions of real context behind us: were our decisions right, and would we make them again?

Today We Review, Not Build

Session 8 held our first Architecture Clinic, after 8 sessions.

Today, after 24 sessions — Observability, CQRS, real Identity, a Service Mesh, closed technical debt, and verified resilience under real load — we do it again, at a very different scale of maturity.

“It works, so it's fine” is not acceptable. “It works, but here is the specific trade-off we accepted, and here is what would make us reconsider it” is what we are after.

Some of you will genuinely ask: did our training platform actually NEED this? That question is not a criticism — it is exactly the judgement this Clinic exists to build.

Phase 3 Recap — What Was Added, and Why Then

S17	Distributed Tracing, Metrics, Structured Logs	<i>Enough cross-service complexity finally existed</i>
S18	CQRS split on product-service	<i>A concrete, described read/write conflict existed</i>
S19	Keycloak Identity Provider	<i>Answered the question Session 3 explicitly deferred</i>
S20	Client Credentials + Resource Server	<i>Closed the loop — JwtAuthFilter formally retired</i>
S21	Istio: mTLS, weighted traffic, mesh resilience	<i>Named, cross-cutting duplication across services</i>
S22	Outbox, Idempotency Keys, API Versioning	<i>Directly closed 3 named debt items</i>
S23	k6 load/stress testing, resilience verified	<i>Turned config into a tested, observed result</i>

Be the Harsh Critic — Raised Stakes

“Imagine a new senior engineer joins the team today, reads the entire codebase and every session pack, and is asked to write an honest architecture review memo before their first week ends.”

A lazy answer: “We added Istio because service meshes are industry standard.”

A good answer: “We added Istio to eliminate cross-cutting duplication, but for our platform's size, the mesh's operational cost may exceed its benefit — a real team our size might reasonably have waited.”

Did Session 18's CQRS Split Earn Its Complexity?

EXPECTED DISCUSSION

Justified given the DESCRIBED dashboard-vs-checkout conflict, but honestly marginal for a training platform without real production read/write load. A real production team should re-verify the pain still exists before copying this pattern elsewhere.

RED FLAG ANSWER

“CQRS should be applied everywhere for good architecture” — no scale/pain justification

Was Session 21's Service Mesh (Istio) the Right Call?

EXPECTED DISCUSSION

Powerful pattern, honestly named trade-off in the session itself: real operational cost (sidecar per pod, control plane, learning curve) for a small, single-language platform. Strong answer: valuable to LEARN, debatable to actually NEED here.

RED FLAG ANSWER

"Istio is industry standard so every platform should use it" — no cost/benefit weighing

Clean Cutover or Parallel Run for Retiring JwtAuthFilter?

EXPECTED DISCUSSION

Correct as taught for a fully-controlled training environment; a real production system with live traffic and external consumers would need to weigh the parallel-run alternative (Session 20's own Engineering Decision) much more seriously.

RED FLAG ANSWER

“There was only one right way to do this” — ignores the session's own stated trade-off

Which Original S8/S12 Debt Items Were NOT Addressed?

EXPECTED DISCUSSION

Open discussion — expect answers referencing in-memory SagaState (S12), dead-letter queue handling maturity, or database migration tooling (Flyway/Liquibase, named since S8 and never implemented). Good answer names a SPECIFIC remaining item and argues its priority.

RED FLAG ANSWER

“All our technical debt is resolved now” — no engineer's platform is ever fully debt-free

The ONE Decision From Sessions 1-16 You'd Make Differently?

EXPECTED DISCUSSION

Open discussion. Strong answers reference something concrete: e.g. introducing structured logging (S17) from Session 1 instead of Session 17, or considering CQRS/Outbox needs earlier given how naturally S7's Saga led to S22's Outbox gap.

RED FLAG ANSWER

"I wouldn't change anything" — no engineer ever, especially not with 23 sessions of hindsight

Growth in Judgement Since Session 8

Compare the **SHAPE** of today's answers to Session 8's Clinic #1 notes.

Instructor Pack's Layer 3 (Judgement) explicitly predicted richer answers here, with more context available.

Session 8 (then)

- “It works, so it's fine” answers common
- Trade-offs named less specifically

Session 24 (now)

- “This was probably premature” willingly said
- Richer trade-off language, real costs named

Pattern-vs-Need — Applied to Every Future Choice

Every Phase 3 addition (CQRS, Mesh, Outbox) was introduced with an explicit justifying problem.

This is the standard to apply to EVERY future architectural choice — including in the Capstone.

Honest Trade-offs, Not Fairy Tales (Instructor Pack P6) — the strongest answers today named a real cost, not just a benefit.

RESOLVED Since Session 8/12



Retry without idempotency on payment calls

Idempotency Keys — Session 22



No API versioning strategy

/api/v1/ + deprecation window — Session 22



Hardcoded JWT secret, no real Identity Provider

Keycloak + OAuth2 — Sessions 19-20



No structured logging

JSON logs + trace correlation — Session 17

STILL OPEN — Carried Into Phase 4 Capstone



In-memory SagaState in OrderSagaOrchestrator (Session 12) — lost on restart



No database migrations (Flyway/Liquibase) — schema managed manually since Session 1



Dead-letter queue handling maturity — DLT events logged (S13) but no automated replay/alerting



PUBLIC_ROUTES-style hardcoded config patterns that may still exist in older session code

NEWLY SURFACED — Populated Live



This slide is intentionally a blank template.

Fill in from the actual class discussion during Questions 1 and 2 (CQRS, Istio) above — each group documents at least one NEWLY SURFACED debt item plus a one-sentence priority justification.

[— blank line for live class input —]

What Capstone Teams Inherit As-Is

Proven, expected to work without re-litigating:

- Config/Eureka/Gateway (S1-3)
- The full Resilience4j stack (S4-5)
- OpenFeign communication (S6)
- The Choreography Saga (S7)
- Docker + K8s + CI/CD + GitOps (S9-16)
- Observability (S17)
- The full security stack (S19-20)

What Remains Open for the Team's Own Judgement

Per today's Design Choice review, per Instructor Pack Section 3.5 (Core vs Advanced Scope):

- Whether to adopt CQRS elsewhere in the platform
- Whether to extend the Service Mesh to every service, or leave it partial
- How to prioritize the STILL OPEN Technical Debt Register v2 items against new Capstone feature work

“Phase 4 begins Session 25 — everything from today's Technical Debt Register v2 is now YOUR call to make, as a team, with a real justification either way.”

Three Questions From Session 23's Homework

1

One architectural decision from Sessions 1-23 you would defend confidently under questioning

2

One decision you are genuinely UNSURE was correct, and why

3

One thing you would do differently if starting the platform over today

Shared aloud during Part 4 of today's session.

8 Sessions · 20 Hours · Enterprise-Grade Capabilities

- ✓ Distributed Tracing, Metrics, Structured Logging (Session 17)
- ✓ CQRS — Command/Query separation with justified scope (Session 18)
- ✓ Real Identity Provider — human login + service-to-service auth, JwtAuthFilter retired (S19-20)
- ✓ Service Mesh — mTLS, native traffic management, mesh-level resilience (Session 21)
- ✓ Outbox, Idempotency, API Versioning — 3 named debt items closed (Session 22)
- ✓ Resilience verified under real load with k6 (Session 23)
- ✓ Architecture Clinic #2 — Technical Debt Register v2 produced (Session 24)

From Review to Building

Phase 3 (S17-24)

- Adding new enterprise capabilities
- Reviewing accumulated decisions
- Learning the trade-offs behind each pattern

Phase 4 (S25-29)

- Building the Capstone — your own decisions now
- Applying judgement, not following a script
- Presenting and defending architecture choices

Everything reviewed today becomes the foundation you build on — or deliberately rebuild — starting Session 25.

Phase 4 Preview — Sessions 25-29

S25	Capstone — Build Day (Offline Anchor Day #4)	<i>Wed 10AM-3PM</i>
------------	--	---------------------

S26	Capstone — Deployment	<i>Online</i>
------------	-----------------------	---------------

S27	Capstone — Observability & Security	<i>Online</i>
------------	-------------------------------------	---------------

S28	Capstone — Testing & Polish	<i>Online</i>
------------	-----------------------------	---------------

S29	Final Presentations & Evaluation	<i>Online</i>
------------	----------------------------------	---------------



Daily Quiz

8 questions · 10 minutes · Reflective, not purely technical

Before Session 25 — Offline Anchor Day #4

Required Before Session 25 (Capstone Build Day):

- Your Capstone team finalized
- Core Scope vs Advanced Scope reviewed (Instructor Pack Section 3.5)
- Technical Debt Register v2 priorities agreed upon as a team
- Full local environment verified working: Docker, Kubernetes, Istio, Keycloak — all reachable



Phase 3 Complete

Session 24 Done [🔗](#) — 8 Sessions · 20 Hours · Enterprise-Grade Platform

Next: Session 25 — Capstone Build Day (Phase 4 Begins)

5-Hour OFFLINE Anchor Day, Wednesday 10:00 AM